

Development of a Python SDK for the Blueframe Embedded Platform

Project Description



03-SEL-BFSDK

Caleb Hansen

Genevieve Kochel

Darron Li

Lucas Phillips

I. Introduction

For many decades, we have relied on the electrical power grid to provide critical, and at times life-saving, infrastructure. The security and reliability of the electrical grid, therefore, is paramount. SEL Blueframe is an operating system and platform designed with this in mind. It is built from a hardened, stripped-down version of the Linux kernel that runs entirely containerized applications, along with many other security features. Blueframe is often run at edge locations; Embedded computing devices such as the SEL 3350 are installed at power substations and run the Blueframe operating system. The goal is extremely high reliability and availability - we should spend minimal time installing patches and updates, and should minimize the chance of system crashes or malicious attacks. As a result of this specialized design, application development for Blueframe is restricted to those with specialized understanding of system APIs and internals. and thus essentially limited to SEL engineers. Our Python SDK project aims to address this gap and create a clean abstraction over the Blueframe system API to make development much more accessible.

II. Background and Related Work

This project sits squarely between the domains of operating systems programming and embedded systems. While the devices that run Blueframe themselves are essentially regular computers with a custom OS, the individual electronic devices (IED) they interact with are often hyper-specialized hardware with a single purpose and limited resources. [3] Creating an SDK for this space requires handling both the high reliability and availability requirements of a robust OS, but also the resource-bound constraints of embedded systems.

Within that domain, a number of existing SEL solutions exist, providing low-level verbose control over the hardware. Users with a deep understanding of the underlying Blueframe system are able to take advantage of this precise control. However, it tends to be unapproachable for novices. Our project is not to recreate these existing API's, as that would be an immense amount of duplicated work. Instead, we are meant to create an SDK to abstract away much of the verbosity and low-level control.

This approach aligns with common practices in modern SDK design, where higher-level abstractions are layered over stable, lower-level systems [1] [2]. Existing SDKs and interface specifications such as OpenAPI and tooling ecosystems like Swagger provide useful reference points for how APIs can be documented, structured, and made discoverable. While these tools are not specific to operating systems or embedded development, they demonstrate widely adopted conventions for API consistency, versioning, and developer experience. In the absence of academic research in this area, developer blog posts, open-source repositories, and real-world SDKs serve as the primary related work and state-of-the-art references.

Completing this project requires acquiring new technical knowledge and skills across several areas. These include deeper familiarity with the existing Blueframe codebase, understanding conventions used by comparable SDKs, and learning best practices for API design in constrained and safety-critical environments. Additionally, the project involves working with containerization, inter-process communication, and permission models that span both the system-level and application-level.

III. Project Overview

The goal of this project is to combine the powerful low-level capabilities of the existing Blueframe APIs with the ease of use and maintainability of the Python language by creating a high-level Python SDK. While the current APIs expose rich functionality and enable fine-grained control over system behavior, they are often verbose, complex, and difficult to approach without deep familiarity with the underlying system. This creates friction when developing new functionality, and increases the likelihood of incorrect or inconsistent usage. Customers and engineers require a simpler solution that better allows for maintainability and software development.

To that end, our SDK is to provide clearer abstractions, safer defaults, and more intuitive interfaces that allow developers to be productive without needing to understand every low-level detail. However, it is crucial that this cleaner approach maintains the same robust and secure qualities of the original API's.

As mentioned in the previous section, we are targeting this project towards engineers who do not have a pure CS background. A major part of effort will need to be focused towards creating documentation that is highly readable and explainable to developers without our background. This will require creating extensive example code, and minimizing jargon and complex design patterns whenever possible.

This is a very sensitive space, and our safety and security requirements are uncompromising. Any abstraction introduced by the SDK must maintain strong guarantees around error handling, authorization, and system state. We will have to be extremely careful and thoughtful when designing our handling for authentication, cookie storage, and other security mechanisms. It is absolutely crucial that these Blueframe systems do not fail.

Performance is also a critical concern. While Blueframe systems typically are not fully embedded, i.e they have powerful modern processors and significant memory, the SDK still cannot introduce any unneeded overhead. Poorly chosen algorithms, excessive memory allocations, or inefficient communication patterns are all to be avoided, as one of the project's objectives is to ensure that improved usability does not come at the cost of performance.

Another important outcome is long-term maintainability. By establishing clear conventions, consistent API patterns, and well-defined abstractions, the project aims to make future development easier for both current and future engineers and customers. This is especially important given that the project is built on an existing codebase and must integrate cleanly with current functionality rather than replacing it.

We will also have access to a pre-existing codebase, and will need to make some judgment calls on its quality. It is likely that we scaffold our project off of it, so we will need to take some time to determine its structure and conventions, and either incorporate those into our project or discard them and replace them with our own. This will require us to extensively practice our skills not just as software developers, but as software architects.

With these requirements in mind, the outcome of this project will be a production quality SDK that can be imported and used by containerized applications running on Blueframe systems. The SDK will provide clear, well-structured abstractions over existing APIs, enabling both SEL engineers and customers to develop new functionality without needing to interact directly with verbose or complex low-level interfaces. It will be designed to be approachable for users with

and without a dedicated computer science background, while preserving the robustness, security, and performance of the existing APIs.

IV. Client and Stakeholder Identification and Preferences

The first and primary stakeholder for our project is the primary client: Joe Stanley and by extension, all of SEL. Joe prefers to have weekly meetings, and expects us to do the architecting, requirements gathering, and development for the project.

Going beyond him, however, our stakeholders include several electrical and computer engineers from Joe's team who will be the primary users of the SDK. These are tech-savvy engineers, but they don't necessarily have a deep computer science and programming background. The SDK must be approachable for these engineers.

Further, future stakeholders could potentially include regular customers of SEL and Blueframe. While our project does not necessarily entail release to the open source community or SEL customers, it could have that potential later on down the line. While these are stakeholders to be aware of, they are not directly related to our project and their preferences can be temporarily ignored.

V. Glossary

OS - Operating System, the low-level software on a computer that enables the hardware to run applications.

IED - Individual Electronic Device.

Blueframe - Custom hardened operating system developed by SEL.

SDK - Software Development Kit, a set of convenient tools and wrappers for developers to write software (typically targeting a specific platform).

SEL - Schweitzer Engineering Laboratories.

VI. References

[1] OpenAPI Initiative, *OpenAPI Specification*, OpenAPI Initiative, 2024. [Online]. Available: <https://www.openapis.org>. Accessed: Feb. 5, 2026.

[2] Swagger (SmartBear Software), *Swagger: API Development Tools*, SmartBear Software, 2024. [Online]. Available: <https://swagger.io>. Accessed: Feb. 6, 2026.

[3] Schweitzer Engineering Laboratories, *Blueframe Platform*, Schweitzer Engineering Laboratories, Inc., Pullman, WA, USA, 2024. [Online]. Available: <https://selinc.com>. Accessed: Feb. 1, 2026.